



รายงาน

เรื่อง การเปรียบเทียบประสิทธิภาพของโมเดล Machine Learning
สำหรับการทำนายผลการเรียนของนักเรียน

จัดทำโดย

นางสาวนัธมนต์ ทองคำ

รหัสนักศึกษา 2310511101113

เสนอ

ผู้ช่วยศาสตราจารย์ ดร.อิทธิพงษ์ เขมะเพชร

คณบดีคณะวิทยาศาสตร์และเทคโนโลยี

รายงานฉบับนี้เป็นส่วนหนึ่งของ SP452 รายวิชาการเรียนรู้ของเครื่อง
ภาคเรียนที่ 2 ปีการศึกษา 2568
คณะวิทยาศาสตร์และเทคโนโลยี สาขาวิชาวิทยาการคอมพิวเตอร์
มหาวิทยาลัยหอการค้าไทย

สารบัญ

เรื่อง

หน้า

1. หลักการและเหตุผล.....	1
2. วัตถุประสงค์.....	2
3. ผลงานที่เกี่ยวข้อง.....	3
4. การพัฒนาระบบ.....	4
5. ผลการวิเคราะห์.....	31
6. ประโยชน์ที่ได้รับ.....	32
7. สรุป.....	33
8. แหล่งอ้างอิง.....	34

1. หลักการและเหตุผล

ในปัจจุบัน การนำข้อมูลมาวิเคราะห์เพื่อสนับสนุนการตัดสินใจในด้านการศึกษาเป็นสิ่งที่มีความสำคัญอย่างมาก เนื่องจากข้อมูลสามารถสะท้อนพฤติกรรมและปัจจัยต่าง ๆ ที่ส่งผลต่อผลการเรียนของนักเรียนได้อย่างชัดเจน การวิเคราะห์ข้อมูลผลการเรียนจึงช่วยให้สามารถเข้าใจแนวโน้มและรูปแบบของผลสัมฤทธิ์ทางการศึกษาได้ดียิ่งขึ้น

โครงการนี้เลือกใช้ชุดข้อมูลเกี่ยวกับผลการเรียนของนักเรียน (Student Performance Dataset) ซึ่งได้มาจากแหล่งข้อมูลเปิด (Open Dataset) จากเว็บไซต์ Kaggle [1] โดยประกอบด้วยตัวแปรสำคัญหลายด้าน เช่น ค่าเฉลี่ยสะสม (GPA) จำนวนครั้งการขาดเรียน (absences) ระยะเวลาในการศึกษา (study time) รวมถึงปัจจัยด้านการสนับสนุนต่าง ๆ โดยข้อมูลดังกล่าวมีลักษณะที่สะท้อนปัจจัยที่ส่งผลต่อผลการเรียนของนักเรียนได้อย่างเหมาะสม และสามารถนำมาใช้ในการวิเคราะห์เพื่อสร้างแบบจำลองการทำนายได้

จากการสำรวจข้อมูลเบื้องต้นในขั้นตอน Data Understanding พบว่าชุดข้อมูลนี้ไม่มีค่าที่หายไป (Missing Values) และข้อมูลทั้งหมดอยู่ในรูปแบบเชิงตัวเลข ทำให้สามารถนำไปใช้กับเทคนิค Machine Learning ได้โดยตรงโดยไม่ต้องผ่านกระบวนการแปลงข้อมูลเพิ่มเติม อีกทั้งยังช่วยลดความซับซ้อนในขั้นตอนการเตรียมข้อมูล (Data Preprocessing)

Machine Learning เป็นเทคนิคที่สามารถเรียนรู้รูปแบบจากข้อมูล และนำไปใช้ในการทำนายผลลัพธ์ในอนาคตได้อย่างมีประสิทธิภาพ ในโครงการนี้จึงได้นำเทคนิค Machine Learning มาประยุกต์ใช้ในการสร้างแบบจำลองเพื่อทำนายผลการเรียนของนักเรียน โดยใช้ตัวแปรเป้าหมาย (Target Variable) คือ GradeClass โดยมีการทดลองใช้โมเดล หลายประเภท เช่น Logistic Regression [2], Decision Tree [3], Random Forest [4] และ Support Vector Machine (SVM) [5] เพื่อเปรียบเทียบประสิทธิภาพของแต่ละโมเดล และเลือกโมเดลที่ให้ผลลัพธ์ที่ดีที่สุด

ดังนั้น โครงการนี้จึงมีความสำคัญในการนำข้อมูลมาวิเคราะห์ร่วมกับเทคนิค Machine Learning เพื่อสร้างแบบจำลองสำหรับการทำนายผลการเรียน ซึ่งสามารถนำไปประยุกต์ใช้ในการวางแผนการเรียน การติดตามผลการเรียนของนักเรียน และสนับสนุนการตัดสินใจในด้านการศึกษาได้อย่างมีประสิทธิภาพมากยิ่งขึ้น

2. วัตถุประสงค์

- 1) เพื่อศึกษาลักษณะของข้อมูลและวิเคราะห์ความสัมพันธ์ของปัจจัยต่าง ๆ ที่ส่งผลต่อผลการเรียนของนักเรียน เพื่อทำความเข้าใจแนวโน้มของข้อมูล และสามารถนำไปใช้ในการวิเคราะห์เชิงลึกได้อย่างเหมาะสม
- 2) เพื่อพัฒนาแบบจำลอง Machine Learning สำหรับการทำนายผลการเรียนของนักเรียน โดยใช้ตัวแปรเป้าหมายคือ GradeClass เพื่อให้สามารถคาดการณ์ผลลัพธ์ได้อย่างมีประสิทธิภาพ และช่วยสนับสนุนการตัดสินใจในด้านการศึกษา
- 3) เพื่อเปรียบเทียบประสิทธิภาพของโมเดล Machine Learning หลายประเภท ได้แก่ Logistic Regression, Decision Tree, Random Forest และ Support Vector Machine (SVM) เพื่อหาความเหมาะสมของแต่ละโมเดล และเลือกแนวทางที่เหมาะสมกับลักษณะของข้อมูล
- 4) เพื่อเลือกโมเดลที่มีประสิทธิภาพสูงสุดและเหมาะสมที่สุดสำหรับการทำนายผลการเรียนของนักเรียนในชุดข้อมูลนี้ เพื่อนำไปประยุกต์ใช้ในการวิเคราะห์และสนับสนุนการวางแผนด้านการศึกษา

3. ผลงานที่เกี่ยวข้อง

จากการศึกษางานที่เกี่ยวข้อง พบว่ามีการนำชุดข้อมูล Student Performance Dataset จากเว็บไซต์ Kaggle [1] มาใช้ในการวิเคราะห์และสร้างแบบจำลอง Machine Learning เพื่อทำนายผลการเรียนของนักเรียนอย่างแพร่หลาย โดยงานส่วนใหญ่มักใช้เทคนิคการจำแนกข้อมูล (Classification) เนื่องจากตัวแปรเป้าหมาย (GradeClass) อยู่ในรูปแบบของการจัดกลุ่มระดับผลการเรียน

โดยจากการสำรวจในส่วนของ Kaggle Code [1] พบว่ามีผู้พัฒนา Notebook หลายรายที่นำชุดข้อมูลนี้ไปใช้ เช่น งานในหัวข้อ “Student Grade Prediction” [1] ซึ่งมีแนวทางการพัฒนาแบบจำลองที่สอดคล้องกับโครงงานนี้ เช่น Logistic Regression [2] หรือ Softmax Regression ซึ่งเหมาะสมสำหรับปัญหาการจำแนกข้อมูลหลายคลาส โดยมีการฝึกโมเดล (Training) ทดสอบโมเดล (Testing) และประเมินผลด้วยค่าความแม่นยำ (Accuracy) เพื่อวัดประสิทธิภาพของแบบจำลอง

นอกจากนี้ ยังมีการวิเคราะห์ความสำคัญของตัวแปร (Feature Importance) และศึกษาความสัมพันธ์ของปัจจัยต่าง ๆ เช่น ค่าเฉลี่ยสะสม (GPA) จำนวนครั้งการขาดเรียน (absences) และระยะเวลาในการศึกษา (study time) ซึ่งพบว่าปัจจัยเหล่านี้มีผลต่อระดับผลการเรียนของนักเรียนอย่างมีนัยสำคัญ และสามารถนำมาใช้เป็นตัวแปรสำคัญในการสร้างแบบจำลองได้

จากแนวทางของผลงานที่เกี่ยวข้องดังกล่าว ผู้จัดทำจึงได้นำแนวคิดการใช้ Machine Learning มาประยุกต์ใช้กับชุดข้อมูลนี้ โดยต่อยอดจากแนวทางเดิมด้วยการทดลองใช้โมเดลหลายประเภท ได้แก่ Logistic Regression [2], Decision Tree [3], Random Forest [4] และ Support Vector Machine (SVM) [5] เพื่อเปรียบเทียบประสิทธิภาพของโมเดล และเลือกโมเดลที่เหมาะสมที่สุดสำหรับการทำนายผลการเรียนของนักเรียน

4. การพัฒนาระบบ

ในการพัฒนาระบบสำหรับการทำนายผลการเรียนของนักเรียนในโครงการนี้ ได้มีการดำเนินงานตามขั้นตอนของกระบวนการ Machine Learning ตั้งแต่การนำเข้าชุดข้อมูล การตรวจสอบและวิเคราะห์ข้อมูลเบื้องต้น (Data Understanding) การสำรวจข้อมูล (Exploratory Data Analysis: EDA) การเตรียมข้อมูล (Data Preparation) ไปจนถึงการสร้างและประเมินผลโมเดล (Modeling and Evaluation) โดยมีการเลือกใช้โมเดล Machine Learning หลายประเภทเพื่อเปรียบเทียบประสิทธิภาพและเลือกโมเดลที่เหมาะสมที่สุดสำหรับชุดข้อมูลนี้

ตัวอย่างขั้นตอนในการพัฒนาโมเดล มีดังนี้

1. การนำเข้าไลบรารีและเตรียมเครื่องมือสำหรับการทำงาน โค้ดที่ใช้ในขั้นตอนนี้มีดังนี้

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

โดยไลบรารีที่ใช้ประกอบด้วย pandas สำหรับจัดการข้อมูลในรูปแบบตาราง, numpy สำหรับคำนวณเชิงตัวเลข, matplotlib และ seaborn สำหรับสร้างกราฟเพื่อแสดงผลการวิเคราะห์ข้อมูล

2. การเชื่อมต่อ Google Drive และกำหนดตำแหน่งโฟลเดอร์โปรเจกต์

```
from google.colab import drive
drive.mount('/content/drive')

import os

project_path = "/content/drive/My Drive/SP452_Natthamon_Final_Project"
os.chdir(project_path)

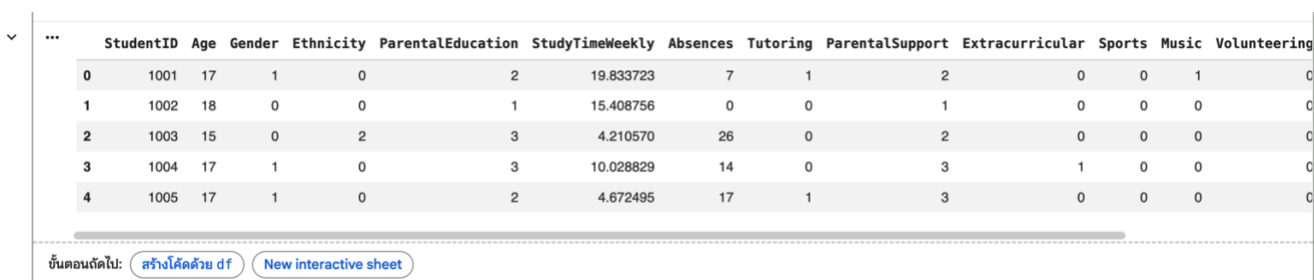
print("Current folder:", os.getcwd())
print("Files:", os.listdir())
```

โค้ดส่วนนี้ใช้สำหรับเชื่อมต่อ Google Colab กับ Google Drive เพื่อให้สามารถเข้าถึงชุดข้อมูลได้ จากนั้นกำหนดตำแหน่งโฟลเดอร์โปรเจกต์และตรวจสอบไฟล์ภายในโฟลเดอร์ เพื่อยืนยันว่าไฟล์ Student_performance_data_.csv อยู่ในตำแหน่งที่ถูกต้องก่อนนำไปใช้งาน

3. การโหลดชุดข้อมูล

```
df = pd.read_csv("Student_performance_data_.csv")
df.head()
```

ขั้นตอนนี้เป็น การอ่านไฟล์ชุดข้อมูลจากไฟล์ .csv โดยใช้คำสั่ง pd.read_csv() และเก็บข้อมูลไว้ในตัวแปร df จากนั้นใช้ df.head() เพื่อแสดงข้อมูล 5 แถวแรกของชุดข้อมูล ทำให้เห็นลักษณะของข้อมูลเบื้องต้น เช่น ชื่อคอลัมน์ ตัวแปรต่าง ๆ และรูปแบบข้อมูลที่จะนำมาใช้ในการวิเคราะห์



	StudentID	Age	Gender	Ethnicity	ParentalEducation	StudyTimeWeekly	Absences	Tutoring	ParentalSupport	Extracurricular	Sports	Music	Volunteering
0	1001	17	1	0	2	19.833723	7	1	2	0	0	1	0
1	1002	18	0	0	1	15.408756	0	0	1	0	0	0	0
2	1003	15	0	2	3	4.210570	26	0	2	0	0	0	0
3	1004	17	1	0	3	10.028829	14	0	3	1	0	0	0
4	1005	17	1	0	2	4.672495	17	1	3	0	0	0	0

รูปที่ 1 ตัวอย่างข้อมูล 5 แถวแรกของชุดข้อมูล Student Performance Dataset

4. การวิเคราะห์ข้อมูลเบื้องต้นและตรวจสอบข้อมูล

```
print("จำนวนแถวและคอลัมน์:", df.shape)
```

```
print("\nชื่อคอลัมน์ทั้งหมด:")
```

```
print(df.columns)
```

```
df.info()
```

```
df.isnull().sum()
```

```
df.describe()
```

ในขั้นตอนนี้เป็น การตรวจสอบโครงสร้างของชุดข้อมูล โดยใช้ df.shape เพื่อดูจำนวนแถวและคอลัมน์ของข้อมูล ใช้ df.columns เพื่อตรวจสอบรายชื่อคอลัมน์ทั้งหมด ใช้ df.info() เพื่อตรวจสอบชนิดข้อมูลของแต่ละตัวแปร และใช้ df.isnull().sum() เพื่อตรวจสอบค่าที่หายไป (NaN) ในชุดข้อมูล จากผลการตรวจสอบพบว่าข้อมูลไม่มีค่าที่หายไป และตัวแปรส่วนใหญ่อยู่ในรูปแบบตัวเลข จึงสามารถ

นำไปใช้กับโมเดล Machine Learning ได้อย่างเหมาะสม นอกจากนี้ใช้ `df.describe()` เพื่อดูค่าสถิติพื้นฐาน เช่น ค่าเฉลี่ย ค่าต่ำสุด ค่าสูงสุด และส่วนเบี่ยงเบนมาตรฐานของข้อมูล

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2392 entries, 0 to 2391
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   StudentID             2392 non-null   int64
1   Age                   2392 non-null   int64
2   Gender                 2392 non-null   int64
3   Ethnicity              2392 non-null   int64
4   ParentalEducation      2392 non-null   int64
5   StudyTimeWeekly        2392 non-null   float64
6   Absences               2392 non-null   int64
7   Tutoring               2392 non-null   int64
8   ParentalSupport        2392 non-null   int64
9   Extracurricular        2392 non-null   int64
10  Sports                 2392 non-null   int64
11  Music                  2392 non-null   int64
12  Volunteering           2392 non-null   int64
13  GPA                    2392 non-null   float64
14  GradeClass             2392 non-null   float64
dtypes: float64(3), int64(12)
memory usage: 280.4 KB
```

รูปที่ 2 ผลลัพธ์จาก `df.info()` แสดงโครงสร้างข้อมูล

```
df.isnull().sum()
```

```
...      0
StudentID  0
Age        0
Gender     0
Ethnicity  0
ParentalEducation  0
StudyTimeWeekly  0
Absences    0
Tutoring    0
ParentalSupport  0
Extracurricular  0
Sports      0
Music       0
Volunteering  0
GPA         0
GradeClass  0
dtype: int64
```

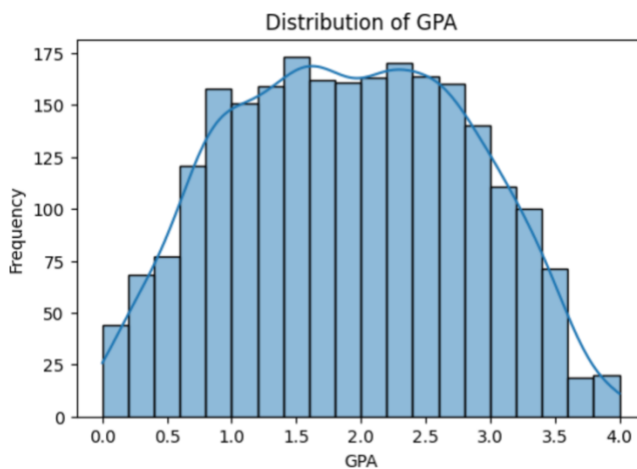
รูปที่ 3 ผลลัพธ์จาก `df.isnull().sum()` แสดงค่าที่หายไป

5. การวิเคราะห์ข้อมูลเชิงสำรวจ (EDA)

5.1 การกระจายของ GPA

```
plt.figure(figsize=(6,4))
sns.histplot(df['GPA'], bins=20, kde=True)
plt.title("Distribution of GPA")
plt.xlabel("GPA")
plt.ylabel("Frequency")
plt.show()
```

จากกราฟพบว่าค่า GPA ของนักเรียนมีการกระจายตัวในช่วงประมาณ 0–4 โดยส่วนใหญ่อยู่ในช่วงระดับปานกลาง แสดงให้เห็นลักษณะการกระจายของข้อมูลก่อนนำไปสร้างโมเดล

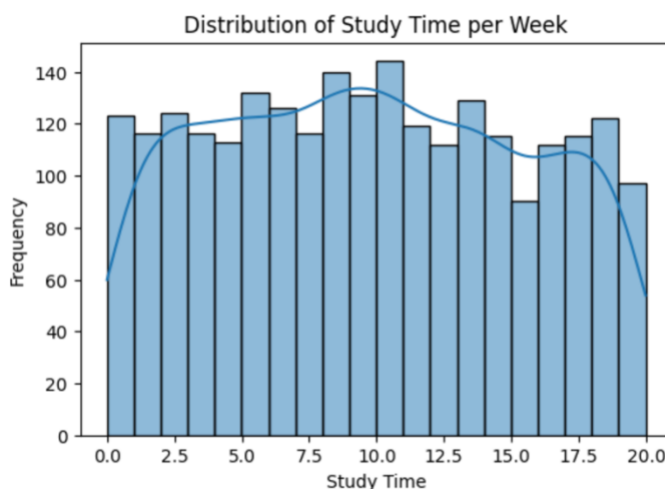


รูปที่ 4 การกระจายตัวของค่า GPA ของนักเรียน

5.2 การกระจายของ Study Time

```
plt.figure(figsize=(6,4))
sns.histplot(df['StudyTimeWeekly'], bins=20, kde=True)
plt.title("Distribution of Study Time per Week")
plt.xlabel("Study Time")
plt.ylabel("Frequency")
plt.show()
```

จากกราฟพบว่าระยะเวลาในการเรียนของนักเรียนมีการกระจายตัวในช่วงประมาณ 0–20 ชั่วโมงต่อสัปดาห์ โดยมีความหนาแน่นในช่วงระดับปานกลาง แสดงให้เห็นว่านักเรียนส่วนใหญ่ใช้เวลาเรียนในระดับทั่วไป ไม่ได้กระจุกตัวเฉพาะช่วงใดช่วงหนึ่ง ซึ่งข้อมูลนี้สามารถนำไปใช้วิเคราะห์ความสัมพันธ์กับผลการเรียนในขั้นตอนถัดไปได้

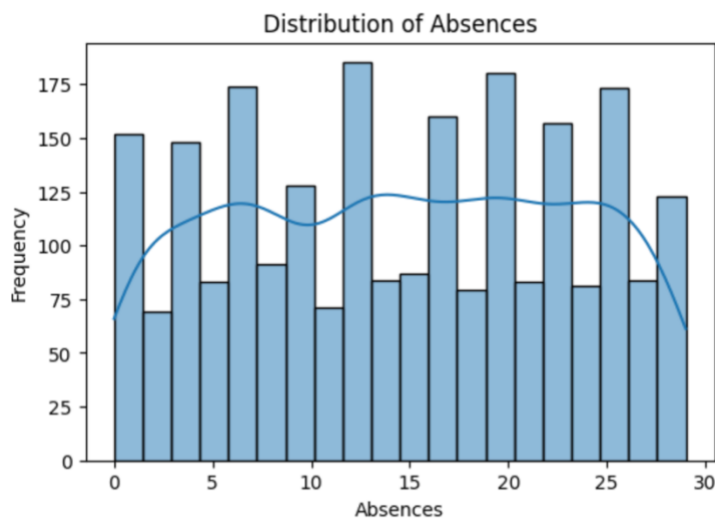


รูปที่ 5 การกระจายตัวของระยะเวลาในการเรียนต่อสัปดาห์

5.3 การกระจายของ Absences

```
plt.figure(figsize=(6,4))
sns.histplot(df['Absences'], bins=20, kde=True)
plt.title("Distribution of Absences")
plt.xlabel("Absences")
plt.ylabel("Frequency")
plt.show()
```

จากกราฟพบว่าจำนวนครั้งการขาดเรียนของนักเรียนมีการกระจายตัวในช่วงประมาณ 0–30 ครั้ง โดยไม่มีการกระจุกตัวในช่วงใดช่วงหนึ่งอย่างชัดเจน แสดงให้เห็นว่านักเรียนมีพฤติกรรมการขาดเรียนที่หลากหลาย ซึ่งอาจส่งผลต่อผลการเรียนและสามารถนำไปวิเคราะห์ร่วมกับตัวแปรอื่นในขั้นตอนถัดไปได้

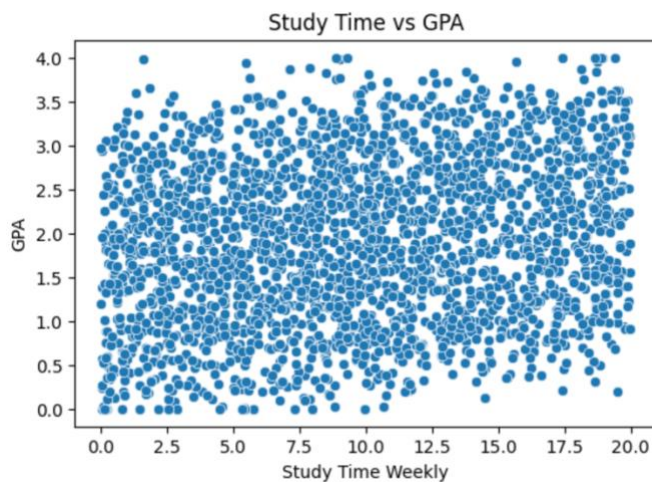


รูปที่ 6 การกระจายตัวของจำนวนครั้งการขาดเรียน

5.4 ความสัมพันธ์ระหว่าง StudyTimeWeekly กับ GPA

```
plt.figure(figsize=(6,4))
sns.scatterplot(x=df['StudyTimeWeekly'], y=df['GPA'])
plt.title("Study Time vs GPA")
plt.xlabel("Study Time Weekly")
plt.ylabel("GPA")
plt.show()
```

จากกราฟ Scatter Plot พบว่าความสัมพันธ์ระหว่างระยะเวลาในการเรียนต่อสัปดาห์กับ GPA ยังไม่ชัดเจน เนื่องจากจุดข้อมูลกระจายตัวค่อนข้างกว้าง อย่างไรก็ตาม อาจสังเกตได้ว่ามีแนวโน้มเล็กน้อยที่นักเรียนที่ใช้เวลาเรียนมากขึ้นจะมีค่า GPA สูงขึ้น ซึ่งแสดงให้เห็นว่าระยะเวลาในการเรียนอาจมีผลต่อผลการเรียน แต่ไม่ใช่ปัจจัยเพียงอย่างเดียว

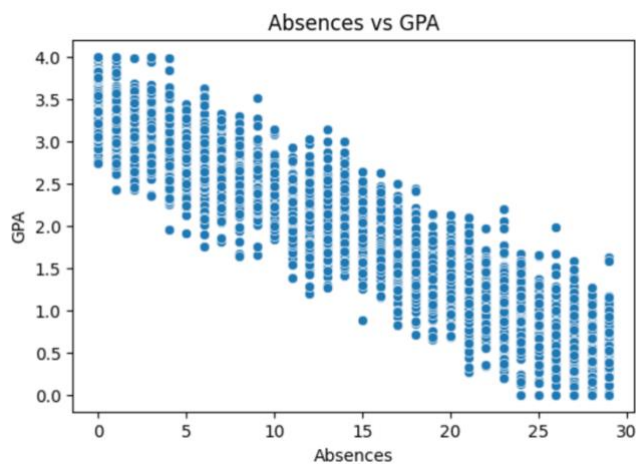


รูปที่ 7 ความสัมพันธ์ระหว่างระยะเวลาในการเรียนต่อสัปดาห์ กับ GPA

5.5 ความสัมพันธ์ระหว่าง Absences กับ GPA

```
plt.figure(figsize=(6,4))
sns.scatterplot(x=df['Absences'], y=df['GPA'])
plt.title("Absences vs GPA")
plt.xlabel("Absences")
plt.ylabel("GPA")
plt.show()
```

จากกราฟ Scatter Plot พบว่าจำนวนครั้งการขาดเรียนมีความสัมพันธ์เชิงลบกับ GPA อย่างชัดเจน โดยเมื่อจำนวนการขาดเรียนเพิ่มขึ้น ค่า GPA มีแนวโน้มลดลง แสดงให้เห็นว่าการขาดเรียนเป็นปัจจัยสำคัญที่ส่งผลต่อผลการเรียนของนักเรียน

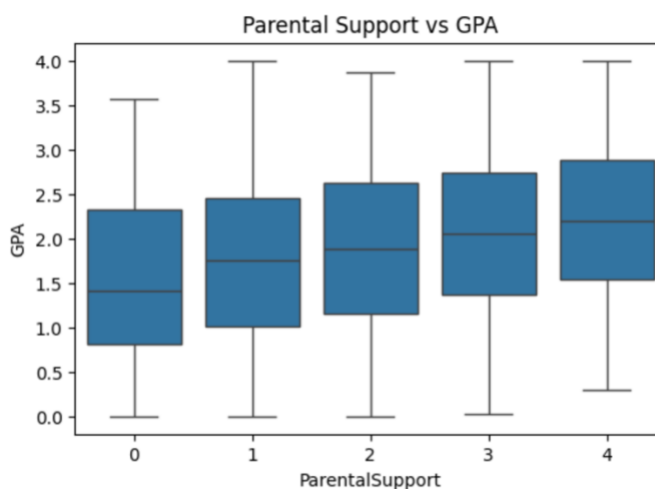


รูปที่ 8 ความสัมพันธ์ระหว่างจำนวนครั้งการขาดเรียน กับ GPA

5.6 ความสัมพันธ์ระหว่าง Parental Support กับ GPA

```
plt.figure(figsize=(6,4))
sns.boxplot(x=df['ParentalSupport'], y=df['GPA'])
plt.title("Parental Support vs GPA")
plt.show()
```

จากกราฟ Boxplot พบว่าระดับการสนับสนุนจากผู้ปกครองมีแนวโน้มสัมพันธ์เชิงบวกกับ GPA โดยนักเรียนที่ได้รับการสนับสนุนในระดับที่สูงขึ้นมักมีค่า GPA สูงขึ้นตามลำดับ นอกจากนี้ยังแสดงให้เห็นการกระจายของข้อมูลในแต่ละกลุ่ม ซึ่งสะท้อนถึงความแตกต่างของผลการเรียนภายใต้ระดับการสนับสนุนที่ต่างกัน



รูปที่ 9 ความสัมพันธ์ระหว่างระดับการสนับสนุนจากผู้ปกครอง กับ GPA

5.7 การวิเคราะห์ความสัมพันธ์ของตัวแปร Correlation Matrix

```
plt.figure(figsize=(8,6))
corr = df.drop(columns=['StudentID']).corr(numeric_only=True)
sns.heatmap(corr,
```

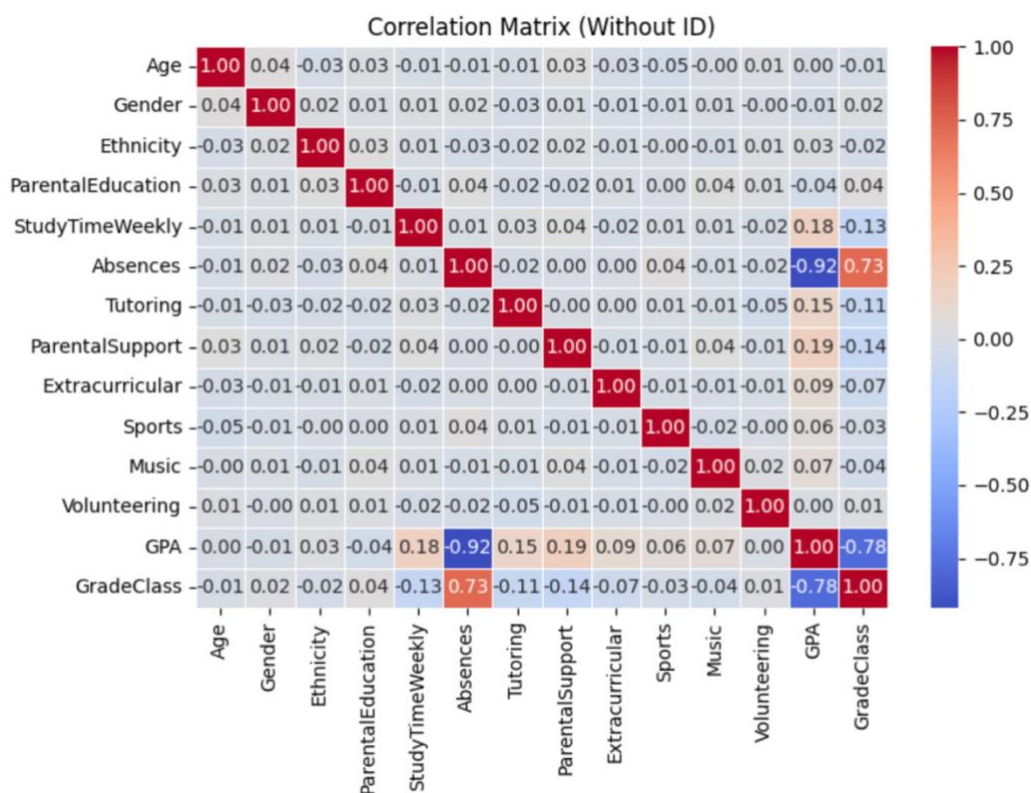
```

annot=True,
cmap='coolwarm',
fmt=".2f",
linewidths=0.5)

plt.title("Correlation Matrix (Without ID)")
plt.tight_layout()
plt.show()

```

จากกราฟ Correlation Matrix พบว่าตัวแปรต่าง ๆ มีความสัมพันธ์กันในระดับที่แตกต่างกัน โดยตัวแปร Absences มีความสัมพันธ์เชิงลบกับ GPA อย่างชัดเจน ขณะที่ StudyTimeWeekly และ ParentalSupport มีความสัมพันธ์เชิงบวกกับ GPA ในระดับหนึ่ง ซึ่งแสดงให้เห็นว่าปัจจัยเหล่านี้มีผลต่อผลการเรียนของนักเรียน และสามารถนำไปใช้เป็นตัวแปรสำคัญในการสร้างแบบจำลอง Machine Learning ได้

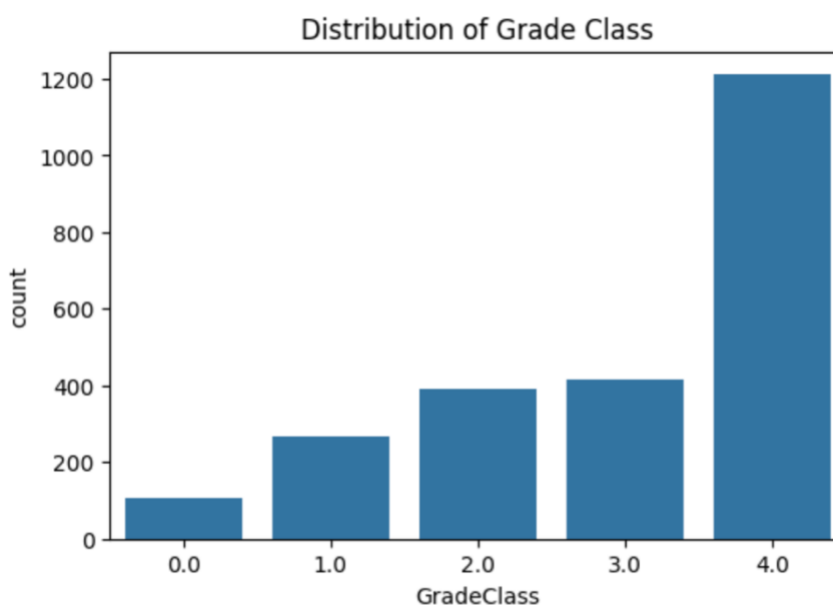


รูปที่ 10 Correlation Matrix แสดงความสัมพันธ์ระหว่างตัวแปร

5.8 การกระจายของ GradeClass

```
plt.figure(figsize=(6,4))
sns.countplot(x=df['GradeClass'])
plt.title("Distribution of Grade Class")
plt.show()
```

กราฟนี้แสดงการกระจายของตัวแปร GradeClass ซึ่งเป็นตัวแปรเป้าหมายในการทำนาย พบว่าจำนวนข้อมูลในแต่ละคลาสมีความแตกต่างกันอย่างชัดเจน แสดงถึงลักษณะข้อมูลที่ไม่สมดุล (Imbalanced Data) ลักษณะดังกล่าวอาจส่งผลกระทบต่อประสิทธิภาพของโมเดล Machine Learning ดังนั้นจึงเลือกใช้การแบ่งข้อมูลแบบ stratified เพื่อรักษาสัดส่วนของแต่ละคลาสให้เหมาะสม



รูปที่ 11 การกระจายของตัวแปรเป้าหมาย (GradeClass)

6. การกำหนดตัวแปรต้นและตัวแปรเป้าหมาย

```
# กำหนดตัวแปรเป้าหมาย (Target)
target = 'GradeClass'

# แยก Features (X) และ Target (y)
X = df.drop(columns=[target, 'StudentID'])
```

```
y = df[target]
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

ขั้นตอนนี้เป็นกำหนัดตัวแปรเป้าหมาย (Target) ของโมเดล คือ GradeClass ซึ่งแสดงระดับผลการเรียนของนักเรียน จากนั้นทำการแยกข้อมูลออกเป็นตัวแปรต้น (X) และตัวแปรเป้าหมาย (y) โดย X ใช้สำหรับการทำนาย และ y คือผลลัพธ์ที่ต้องการทำนาย นอกจากนี้ได้คัดคอลัมน์ StudentID ออกเนื่องจากเป็นเพียงรหัสระบุตัวบุคคลและไม่มีควำมหมายเชิงวิเคราะห์ต่อผลการเรียน จากผลลัพธ์พบว่า X มี 13 ตัวแปร และข้อมูลทั้งหมด 2,392 แถว ซึ่งเพียงพอสำหรับการนำไปสร้างโมเดล Machine Learning

Shape of X: (2392, 13)
Shape of y: (2392,)

รูปที่ 12 การแสดงขนาดของชุดข้อมูลหลังการแยกตัวแปร (Features และ Target)

7. การแบ่งข้อมูล Train และ Test

```
from sklearn.model_selection import train_test_split
# แบ่งข้อมูล train และ test
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
print("Train target distribution:\n", y_train.value_counts(normalize=True))
print("Test target distribution:\n", y_test.value_counts(normalize=True))
```


โค้ดนี้ใช้แบ่งข้อมูลออกเป็นชุดฝึกสอนและชุดทดสอบ โดยกำหนดให้ข้อมูลทดสอบมีขนาด 20% ของข้อมูลทั้งหมด และข้อมูลฝึกสอนมีขนาด 80% ใช้ random_state=42 เพื่อให้ผลการแบ่งข้อมูลสามารถทำซ้ำได้ และใช้ stratify=y เพื่อรักษาสัดส่วนของแต่ละคลาสในตัวแปร GradeClass ให้ใกล้เคียงกันทั้งในชุด train และ test ซึ่งช่วยให้การประเมินผลโมเดลมีความน่าเชื่อถือมากขึ้น

```
Train shape: (1913, 13)
Test shape: (479, 13)
Train target distribution:
GradeClass
4.0    0.506012
3.0    0.173027
2.0    0.163617
1.0    0.112389
0.0    0.044956
Name: proportion, dtype: float64
Test target distribution:
GradeClass
4.0    0.507307
3.0    0.173278
2.0    0.162839
1.0    0.112735
0.0    0.043841
Name: proportion, dtype: float64
```

รูปที่ 13 การแบ่งข้อมูล Train-Test แบบรักษาสัดส่วนคลาส

8. Algorithm ที่เลือกใช้

ในโครงงานนี้เลือกใช้โมเดล Machine Learning สำหรับปัญหา Classification จำนวน 4 โมเดล ได้แก่

1. การสร้างและประเมินผล Logistic Regression

1.1 แนวคิดของโมเดล

เป็นโมเดลพื้นฐานสำหรับงาน Classification ที่ใช้ในการจำแนกข้อมูล โดยในโครงงานนี้ถูกใช้เป็น baseline model เพื่อเปรียบเทียบประสิทธิภาพกับโมเดลอื่น เนื่องจากมีโครงสร้างเรียบง่ายและสามารถตีความผลลัพธ์ได้ง่าย

1.2 การสร้างโมเดล

```

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

# สร้าง pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=2000, random_state=42))
])

# train
pipeline.fit(X_train, y_train)

# predict
y_pred = pipeline.predict(X_test)

# evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))


from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# สร้าง confusion matrix
cm = confusion_matrix(y_test, y_pred)

# แสดงผลเป็นกราฟ
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Logistic Regression")
plt.show()

```

1.3 ผลลัพธ์ของโมเดล

จากการทดลอง พบว่าโมเดลมีค่า Accuracy เท่ากับ **0.8017 (ประมาณ 80%)** พร้อมทั้งมีการแสดงผลในรูปแบบ Classification Report และ Confusion Matrix เพื่อวิเคราะห์รายละเอียดของการจำแนกในแต่ละคลาส

จากผลลัพธ์ของ Logistic Regression สามารถวิเคราะห์ได้ดังนี้:

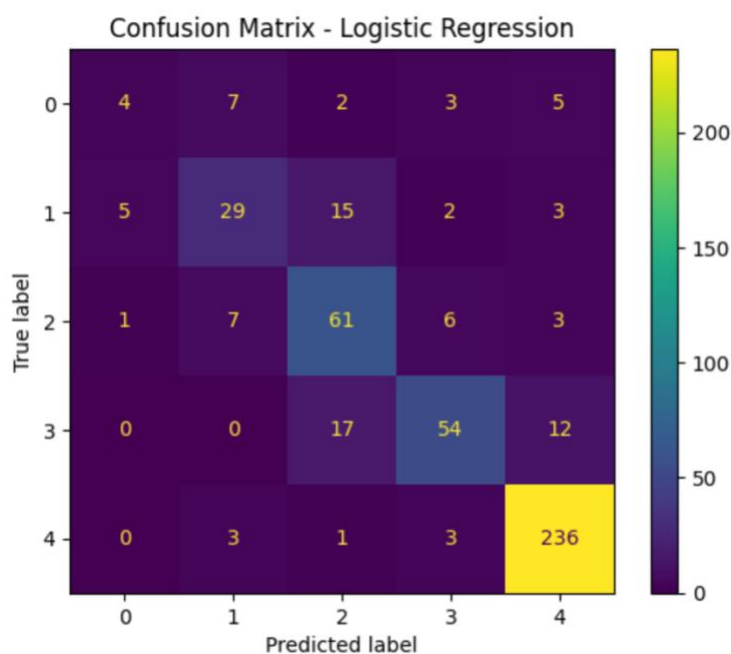
- โมเดลมีค่า Accuracy ประมาณ **80%** ซึ่งถือว่าอยู่ในระดับที่ดีสำหรับโมเดลพื้นฐาน
- เมื่อพิจารณา Classification Report พบว่า:
 - คลาส **4** มีค่า precision และ recall สูงมาก (0.91 และ 0.97) แสดงว่าโมเดลสามารถทำนายคลาสนี้ได้แม่นยำ
 - คลาส **2 และ 3** มีผลลัพธ์อยู่ในระดับดี (f1-score ประมาณ 0.70+)
 - อย่างไรก็ตาม คลาส **0 และ 1** มีค่า recall และ f1-score ต่ำ โดยเฉพาะคลาส 0 (f1-score = 0.26) แสดงว่าโมเดลยังมีข้อจำกัดในการจำแนกคลาสนี้ที่มีจำนวนน้อย
- จาก Confusion Matrix พบว่า:
 - โมเดลมีแนวโน้มทำนายไปยังคลาสนี้ที่มีจำนวนข้อมูลมาก (โดยเฉพาะคลาส 4)
 - มีการสับสนระหว่างคลาสนี้ที่มีค่าระดับใกล้เคียงกัน เช่น 2 กับ 3 และ 3 กับ 4

Accuracy: 0.8016701461377871

Classification Report:

	precision	recall	f1-score	support
0.0	0.40	0.19	0.26	21
1.0	0.63	0.54	0.58	54
2.0	0.64	0.78	0.70	78
3.0	0.79	0.65	0.72	83
4.0	0.91	0.97	0.94	243
accuracy			0.80	479
macro avg	0.67	0.63	0.64	479
weighted avg	0.79	0.80	0.79	479

รูปที่ 14 ผลการประเมินโมเดล Logistic Regression (Accuracy และ Classification Report)



รูปที่ 15 Confusion Matrix ของโมเดล Logistic Regression

2. การสร้างและประเมินผล Decision Tree

2.1 แนวคิดของโมเดล

เป็นโมเดลสำหรับงาน Classification ที่ทำงานโดยการแบ่งข้อมูลออกเป็นกลุ่มย่อยตามเงื่อนไขของตัวแปร (feature) ในรูปแบบโครงสร้างต้นไม้ (tree structure) ซึ่งช่วยให้สามารถเข้าใจและอธิบายกระบวนการตัดสินใจของโมเดลได้อย่างชัดเจน

ในโครงงานนี้ซึ่งเป็นการทำนายผลการเรียนของนักเรียน (Student Performance Prediction) โมเดล Decision Tree [3] ถูกนำมาใช้เพื่อเรียนรู้รูปแบบความสัมพันธ์ของตัวแปรต่าง ๆ เช่น GPA, จำนวนครั้งที่ขาดเรียน (absences) และปัจจัยด้านการเรียนอื่น ๆ โดยโมเดลสามารถแบ่งกลุ่มข้อมูลตามเงื่อนไขที่เหมาะสม เพื่อจำแนกระดับผลการเรียน (GradeClass)

2.2 การสร้างโมเดล

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```

# สร้างโมเดล
dt_model = DecisionTreeClassifier(random_state=42)

# train
dt_model.fit(X_train, y_train)

# predict
y_pred_dt = dt_model.predict(X_test)

# evaluate
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# สร้าง confusion matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)

# แสดงผลเป็นกราฟ
disp = ConfusionMatrixDisplay(confusion_matrix=cm_dt)
disp.plot()
plt.title("Confusion Matrix - Decision Tree")
plt.show()

```

2.3 ผลลัพธ์ของโมเดล

จากการทดลอง พบว่าโมเดล Decision Tree มีค่า **Accuracy เท่ากับ 0.8685 (ประมาณ 86.85%)** ซึ่งถือว่าสูงกว่า Logistic Regression อย่างชัดเจน แสดงให้เห็นว่าโมเดลสามารถเรียนรู้รูปแบบของข้อมูลได้ดีขึ้น โดยเฉพาะในกรณีที่ข้อมูลมีความสัมพันธ์แบบไม่เป็นเชิงเส้น

จากผลลัพธ์สามารถวิเคราะห์ได้ดังนี้:

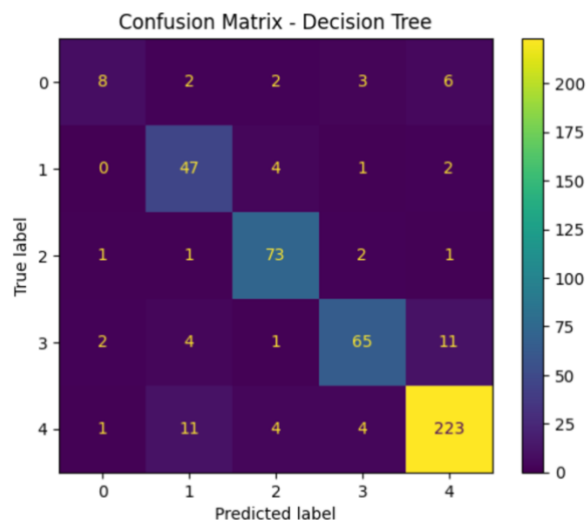
- โมเดลมีความสามารถในการจำแนกข้อมูลโดยรวมได้ดี (Accuracy ~87%)
ซึ่งสูงกว่าโมเดลพื้นฐาน (Logistic Regression ~80%) แสดงให้เห็นว่า Decision Tree สามารถจับ pattern ของข้อมูลได้ซับซ้อนกว่า
- โดยจาก Classification Report พบว่า class 4 มีค่า f1-score สูงถึง 0.92 และ class 2 ประมาณ 0.90 ขณะที่ class 0 ยังมีค่า f1-score ต่ำ (~0.48) แสดงให้เห็นว่าปัญหา class imbalance ยังคงมีผลต่อโมเดล
- Decision Tree สามารถจัดการกับ feature ที่มีความสัมพันธ์ซับซ้อน เช่น GPA, absences และปัจจัยด้านการเรียน ได้ดีกว่า
เนื่องจากการแบ่งเงื่อนไขแบบลำดับขั้น (hierarchical splitting)
- โมเดลมีแนวโน้มให้ผลลัพธ์ที่ดีขึ้นในหลายคลาส โดยเฉพาะคลาสที่มีข้อมูลจำนวนมาก ซึ่งช่วยเพิ่มค่า Accuracy โดยรวม
- อย่างไรก็ตาม Decision Tree มีข้อจำกัดคือ:
 - มีโอกาสเกิด **Overfitting** ได้ง่าย หากต้นไม้ลึกเกินไป
 - ยังคงได้รับผลกระทบจากปัญหา **Imbalanced Data** ทำให้บางคลาสที่มีข้อมูลน้อยอาจถูกทำนายได้ไม่ดี

Accuracy: 0.8684759916492694

Classification Report:

	precision	recall	f1-score	support
0.0	0.67	0.38	0.48	21
1.0	0.72	0.87	0.79	54
2.0	0.87	0.94	0.90	78
3.0	0.87	0.78	0.82	83
4.0	0.92	0.92	0.92	243
accuracy			0.87	479
macro avg	0.81	0.78	0.78	479
weighted avg	0.87	0.87	0.87	479

รูปที่ 16 ผลการประเมินโมเดล Decision Tree (Accuracy และ Classification Report)



รูปที่ 17 Confusion Matrix ของโมเดล Decision Tree

3. การสร้างและประเมินผล **Random Forest**

3.1 แนวคิดของโมเดล

เป็นโมเดลประเภท **Ensemble Learning** ที่เกิดจากการรวม Decision Tree [3] หลายต้นเข้าด้วยกัน โดยแต่ละต้นจะเรียนรู้จากข้อมูลที่ถูกสุ่ม (random sampling) และใช้ subset ของ feature ที่แตกต่างกัน ส่งผลให้โมเดลมีความแม่นยำและความเสถียรมากขึ้นเมื่อเทียบกับ Decision Tree เพียงต้นเดียว

ในโครงงานนี้ซึ่งเป็นการทำนายผลการเรียนของนักเรียน (Student Performance Prediction) โมเดล Random Forest [4] ถูกนำมาใช้เพื่อเรียนรู้ความสัมพันธ์ของตัวแปรต่าง ๆ เช่น GPA, จำนวนครั้งที่ขาดเรียน (absences) และปัจจัยด้านการเรียนอื่น ๆ โดยอาศัยการรวมผลการตัดสินใจจากหลายต้นไม้เพื่อจำแนกระดับผลการเรียน (GradeClass) ได้อย่างมีประสิทธิภาพมากขึ้น

ข้อดีสำคัญของโมเดลนี้คือสามารถลดปัญหา **Overfitting** ที่มักเกิดใน Decision Tree และสามารถจับ pattern ของข้อมูลที่ซับซ้อนได้ดี เหมาะสำหรับข้อมูลที่มีความสัมพันธ์แบบไม่เป็นเชิงเส้น

3.2 การสร้างโมเดล

```
from sklearn.ensemble import RandomForestClassifier
```

```

from sklearn.metrics import accuracy_score, classification_report

# สร้างโมเดล
rf_model = RandomForestClassifier(random_state=42)

# train
rf_model.fit(X_train, y_train)

# predict
y_pred_rf = rf_model.predict(X_test)

# evaluate
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# confusion matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf)
disp.plot()
plt.title("Confusion Matrix - Random Forest")
plt.show()

```

3.3 ผลลัพธ์ของโมเดล

จากการทดลอง พบว่าโมเดล **Random Forest** มีค่า **Accuracy** เท่ากับประมาณ **0.9123 (หรือ 91.23%)** ซึ่งสูงกว่า Decision Tree (~86.85%) และ Logistic Regression (~80%) อย่างชัดเจน แสดงให้เห็นว่าโมเดลสามารถเรียนรู้รูปแบบของข้อมูลได้ดีกว่า

จากผลลัพธ์สามารถวิเคราะห์ได้ดังนี้:

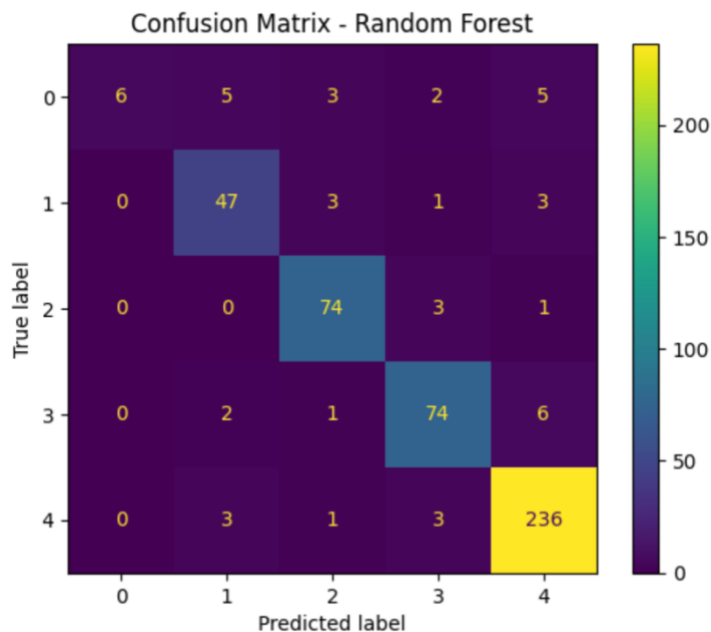
- โมเดลมีความสามารถในการจำแนกข้อมูลโดยรวมได้ดีมาก (Accuracy ~91%)
ซึ่งสูงกว่าโมเดลก่อนหน้านี้ แสดงให้เห็นว่าการใช้ Ensemble ช่วยเพิ่มประสิทธิภาพของโมเดลได้จริง
- จาก **Classification Report** พบว่าในหลาย class มีค่า precision, recall และ f1-score อยู่ในระดับสูงและสมดุลมากขึ้น
แสดงให้เห็นว่าโมเดลมีความสามารถในการทำนายได้ดีในหลายกลุ่มข้อมูล
- โมเดลสามารถจัดการกับ feature ที่มีความสัมพันธ์ซับซ้อน เช่น GPA, absences และปัจจัยด้านการเรียนได้ดีกว่า
เนื่องจากการรวมผลจากหลาย Decision Tree (bagging)
- จาก **Confusion Matrix** จะเห็นว่าค่าบนเส้นทแยงมุมมีค่าสูงขึ้นเมื่อเทียบกับ Decision Tree
แสดงว่าโมเดลทำนายถูกในแต่ละ class มากขึ้น และมีความผิดพลาดลดลง
- โมเดลมีแนวโน้มลดปัญหา **Overfitting** เมื่อเทียบกับ Decision Tree
เนื่องจากใช้หลายต้นไม้ช่วยเฉลี่ยผลลัพธ์
- อย่างไรก็ตาม โมเดลยังคงได้รับผลกระทบจาก **Imbalanced Data** ในบาง class
ทำให้บางกลุ่มที่มีข้อมูลน้อยอาจยังมีความคลาดเคลื่อนในการทำนาย

Accuracy: 0.9123173277661796

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	0.29	0.44	21
1.0	0.82	0.87	0.85	54
2.0	0.90	0.95	0.93	78
3.0	0.89	0.89	0.89	83
4.0	0.94	0.97	0.96	243
accuracy			0.91	479
macro avg	0.91	0.79	0.81	479
weighted avg	0.92	0.91	0.90	479

รูปที่ 18 ผลการประเมินโมเดล Random Forest (Accuracy และ Classification Report)



รูปที่ 19 Confusion Matrix ของโมเดล Random Forest

4. การสร้างและประเมินผล SVM (Support Vector Machine)

4.1 แนวคิดของโมเดล

เป็นโมเดลสำหรับงาน Classification ที่ทำงานโดยการหาเส้นแบ่ง (hyperplane) ที่สามารถแยกข้อมูลแต่ละ class ออกจากกันได้ดีที่สุด โดยพยายามเพิ่มระยะห่าง (margin) ระหว่างข้อมูลของแต่ละกลุ่มให้มากที่สุด

ในโครงงานนี้ซึ่งเป็นการทำนายผลการเรียนของนักเรียน (Student Performance Prediction) โมเดล SVM [5] ถูกนำมาใช้เพื่อเรียนรู้ความสัมพันธ์ของตัวแปรต่าง ๆ เช่น GPA, จำนวนครั้งที่ขาดเรียน (absences) และปัจจัยด้านการเรียนอื่น ๆ เพื่อจำแนกระดับผลการเรียน (GradeClass)

เนื่องจาก SVM มีความไวต่อขนาดของข้อมูล จึงใช้ **StandardScaler** ร่วมกับ **Pipeline** เพื่อปรับขนาดข้อมูลก่อนเข้าสู่โมเดล ทำให้การเรียนรู้มีประสิทธิภาพมากขึ้น โดยเฉพาะในกรณีที่ feature มีช่วงค่า (scale) ที่แตกต่างกัน

4.2 การสร้างโมเดล

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

svm_model = Pipeline([
    ('scaler', StandardScaler()),
    ('model', SVC())
])

svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm))

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm_svm = confusion_matrix(y_test, y_pred_svm)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_svm)
disp.plot()
plt.title("Confusion Matrix - SVM")
plt.show()

```

4.3 ผลลัพธ์ของโมเดล

จากการทดลอง พบว่าโมเดล SVM มีค่า Accuracy เท่ากับประมาณ 0.795 (หรือ 79.5%) ซึ่งต่ำกว่า Logistic Regression (~80%), Decision Tree (~86.85%) และ Random Forest (~91.23%) แสดงให้เห็นว่าโมเดลยังไม่สามารถเรียนรู้รูปแบบของข้อมูลได้ดีเท่ากับโมเดลอื่นในโครงการนี้

จากผลลัพธ์สามารถวิเคราะห์ได้ดังนี้:

- โมเดลมีความสามารถในการจำแนกข้อมูลโดยรวมอยู่ในระดับปานกลาง (Accuracy ~79%) แสดงให้เห็นว่า SVM อาจไม่เหมาะกับลักษณะข้อมูลชุดนี้เท่ากับโมเดลอื่น
- จาก **Classification Report** พบว่า
 - class 4 มีค่า f1-score สูง (~0.93) แสดงว่าทำนายกลุ่มที่มีข้อมูลจำนวนมากได้ดี
 - class 2 และ class 3 อยู่ในระดับปานกลาง (~0.68–0.69)
 - class 0 มีค่า f1-score ต่ำ (~0.31) แสดงให้เห็นว่ามีความยากในการจำแนก และได้รับผลกระทบ

จาก class imbalance

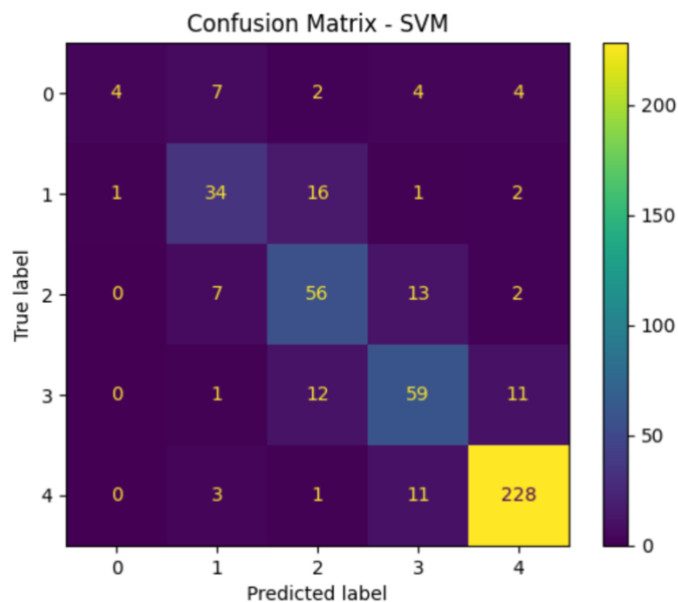
- จาก **Confusion Matrix** จะเห็นว่ามีการทำนายผิดระหว่าง class ใกล้เคียงกันค่อนข้างมาก เช่น class 1 และ class 2 หรือ class 2 และ class 3 แสดงให้เห็นว่า SVM แยกขอบเขตของข้อมูลบางกลุ่มได้ไม่ชัดเจน
- โมเดลมีข้อจำกัดในการจับ pattern ที่ซับซ้อนของข้อมูล โดยเฉพาะเมื่อไม่ได้ปรับ parameter (tuning) เช่น kernel หรือค่า C และ gamma
- อย่างไรก็ตาม SVM มีข้อดีคือสามารถทำงานได้ดีในกรณีข้อมูลที่มีขนาดเล็กและมี margin ชัดเจน แต่ใน dataset นี้ที่มีหลาย class และ pattern ซับซ้อน โมเดลจึงทำได้ไม่ดีเท่าที่ควร

SVM Accuracy: 0.7954070981210856

Classification Report:

	precision	recall	f1-score	support
0.0	0.80	0.19	0.31	21
1.0	0.65	0.63	0.64	54
2.0	0.64	0.72	0.68	78
3.0	0.67	0.71	0.69	83
4.0	0.92	0.94	0.93	243
accuracy			0.80	479
macro avg	0.74	0.64	0.65	479
weighted avg	0.80	0.80	0.79	479

รูปที่ 20 ผลการประเมินโมเดล SVM (Accuracy และ Classification Report)



รูปที่ 21 Confusion Matrix ของโมเดล SVM

9. การเปรียบเทียบประสิทธิภาพของโมเดล

```
import pandas as pd

results = pd.DataFrame({
    'Model': ['Logistic Regression', 'Decision Tree', 'Random Forest', 'SVM'],
    'Accuracy': [
        accuracy_score(y_test, y_pred),
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_rf),
        accuracy_score(y_test, y_pred_svm)
    ]
})

print(results)
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.barplot(x='Model', y='Accuracy', data=results)
plt.title("Model Comparison")
plt.xticks(rotation=30)
plt.show()
```

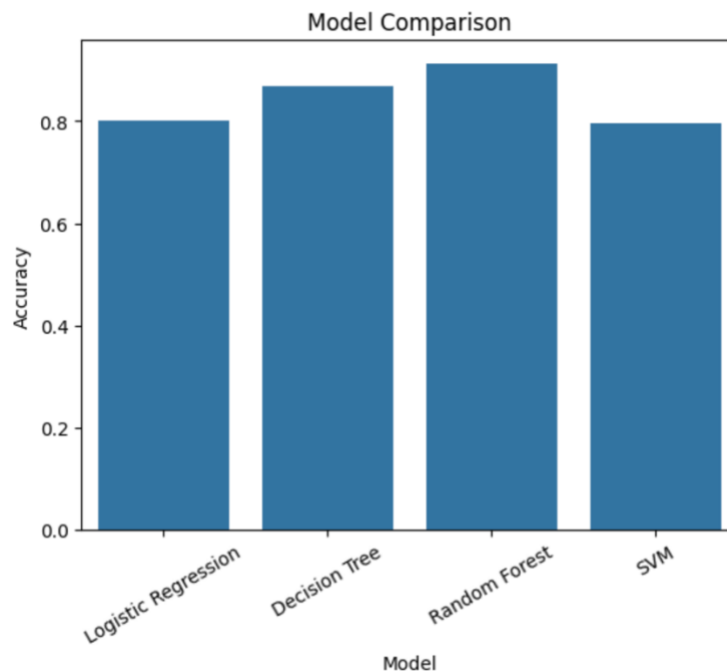
โค้ดในส่วนนี้ใช้สร้างตารางเปรียบเทียบประสิทธิภาพของโมเดล Machine Learning โดยใช้ค่า Accuracy ของแต่ละโมเดล ได้แก่ Logistic Regression, Decision Tree, Random Forest และ SVM จากนั้นแสดงผลในรูปแบบตารางและกราฟแท่ง (Bar Plot) เพื่อให้เห็นความแตกต่างของแต่ละโมเดลได้ชัดเจน

จากผลลัพธ์ในตารางและกราฟ พบว่า

- Random Forest มีค่า Accuracy สูงที่สุด (0.91)
- รองลงมาคือ Decision Tree (0.87)
- Logistic Regression (0.80)
- และ SVM (0.79)

	Model	Accuracy
0	Logistic Regression	0.801670
1	Decision Tree	0.868476
2	Random Forest	0.912317
3	SVM	0.795407

รูปที่ 22 ผลการเปรียบเทียบค่า Accuracy ของแต่ละโมเดล



รูปที่ 23 กราฟเปรียบเทียบประสิทธิภาพของโมเดล (Model Comparison)

10. Confusion Matrix ของโมเดลที่ดีที่สุด

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf)
disp.plot()
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

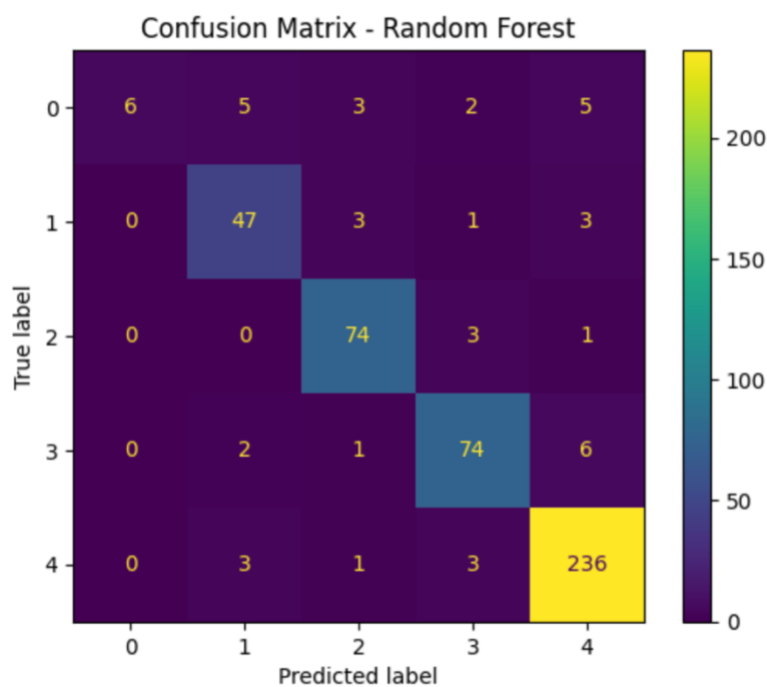
โค้ดในส่วนนี้ใช้สร้าง Confusion Matrix ของโมเดล Random Forest ซึ่งเป็นโมเดลที่มีประสิทธิภาพดีที่สุด โดยใช้ฟังก์ชัน `confusion_matrix` และ `ConfusionMatrixDisplay` เพื่อแสดงผลในรูปแบบกราฟ

จากผลลัพธ์ Confusion Matrix พบว่า

- ค่าบนแนวทแยงมุม (diagonal) มีค่าสูง แสดงว่าโมเดลทำนายได้ถูกต้องในหลายคลาส

- โดยเฉพาะ class 4 มีจำนวนการทำนายถูกต้องสูง (236 ตัวอย่าง)
- class 2 และ class 3 มีค่าถูกต้องในระดับดี
- ในขณะที่ class 0 มีการทำนายผิดมากกว่าคลาสอื่น

ซึ่งแสดงให้เห็นรูปแบบการทำนายของโมเดลในแต่ละคลาส



รูปที่ 24 Confusion Matrix ของโมเดลที่ดีที่สุด (Random Forest)

5. ผลการวิเคราะห์

จากผลการพัฒนาโมเดล Machine Learning เพื่อทำนายผลการเรียนของนักเรียน โดยใช้โมเดล 4 แบบ ได้แก่ Logistic Regression, Decision Tree, Random Forest และ SVM พบว่าโมเดล Random Forest ให้ประสิทธิภาพดีที่สุด โดยมีค่า Accuracy สูงสุดประมาณ 0.91 รองลงมาคือ Decision Tree (0.87), Logistic Regression (0.80) และ SVM (0.79)

ผลลัพธ์นี้สะท้อนให้เห็นว่าโมเดลประเภท Ensemble อย่าง Random Forest มีความสามารถในการเรียนรู้ข้อมูลได้ดีกว่า เนื่องจากการรวม Decision Tree หลายต้น ทำให้ลดปัญหา overfitting และสามารถจับความสัมพันธ์ของข้อมูลที่ซับซ้อนและไม่เป็นเชิงเส้นได้ดี ซึ่งเหมาะกับลักษณะของ dataset นี้

เมื่อพิจารณาจาก Confusion Matrix พบว่าโมเดลสามารถทำนายข้อมูลได้ถูกต้องในหลายคลาส โดยค่าบนแนวทแยงมุมมีค่าสูง โดยเฉพาะ class 4 ที่มีจำนวนการทำนายถูกต้องมาก (236 ตัวอย่าง) แสดงว่าคลาสนี้มี pattern ชัดเจน และโมเดลสามารถเรียนรู้ได้ดี นอกจากนี้ class 2 และ class 3 ก็มีความแม่นยำในระดับที่ดีเช่นกัน

อย่างไรก็ตาม พบว่า class 0 และบางส่วนของ class 1 มีการทำนายผิดมากกว่าคลาสอื่น ซึ่งสะท้อนถึงปัญหา class imbalance เนื่องจากข้อมูลในแต่ละคลาสมีจำนวนไม่เท่ากัน ทำให้โมเดลมีแนวโน้มทำนายคลาสที่มีข้อมูลมากได้ดีกว่า

โดยรวมแล้วสามารถสรุปได้ว่า Random Forest เป็นโมเดลที่เหมาะสมที่สุดสำหรับ dataset นี้ เนื่องจากให้ความแม่นยำสูง มีความเสถียร และสามารถจัดการกับความซับซ้อนของข้อมูลได้ดีกว่าโมเดลอื่น

ตารางที่ 1 ผลการเปรียบเทียบประสิทธิภาพของโมเดล

Model	Accuracy
Logistic Regression	0.8017
Decision Tree	0.8685
Random Forest	0.9123
SVM	0.7954

6. ประโยชน์ที่ได้รับ

จากการพัฒนาโมเดล Machine Learning เพื่อทำนายผลการเรียนของนักเรียน สามารถสรุปประโยชน์ที่ได้รับจากโครงการนี้ได้ดังนี้

1) ช่วยในการคาดการณ์ผลการเรียนของนักเรียน

โมเดลสามารถทำนาย GradeClass ของนักเรียนได้ล่วงหน้า ทำให้สามารถประเมินแนวโน้มผลการเรียนและวางแผนการเรียนได้อย่างเหมาะสม

2) สนับสนุนการตัดสินใจทางการศึกษา

ผู้สอนหรือผู้ดูแลสามารถนำผลการทำนายไปใช้ในการวิเคราะห์นักเรียนที่มีความเสี่ยง เช่น นักเรียนที่มีโอกาสได้เกรดต่ำ และสามารถให้คำแนะนำหรือช่วยเหลือได้ทันเวลาที่

3) ช่วยวิเคราะห์ปัจจัยที่มีผลต่อผลการเรียน

จากการวิเคราะห์ข้อมูล พบว่าปัจจัย เช่น GPA, จำนวนการขาดเรียน (absences), และเวลาในการเรียน (study time) มีผลต่อผลการเรียน ซึ่งสามารถนำไปใช้ปรับปรุงพฤติกรรมการเรียนได้

4) เป็นแนวทางในการพัฒนาโมเดล Machine Learning ในอนาคต

โครงการนี้สามารถใช้เป็นพื้นฐานในการพัฒนาโมเดลที่มีความแม่นยำมากขึ้น หรือทดลองใช้เทคนิคอื่น เช่น Deep Learning หรือการปรับแต่ง Hyperparameter

5) สามารถนำไปประยุกต์ใช้กับระบบจริงได้

โมเดลสามารถนำไปพัฒนาเป็นระบบช่วยแนะนำ (Recommendation System) หรือระบบติดตามผลการเรียนของนักเรียนในสถาบันการศึกษาได้

7. สรุป

โครงการนี้เป็นการประยุกต์ใช้เทคนิค Machine Learning เพื่อวิเคราะห์และทำนายผลการเรียนของนักเรียน โดยอาศัยชุดข้อมูล Student Performance Dataset ซึ่งมีตัวแปรที่สะท้อนพฤติกรรมและปัจจัยด้านการเรียนหลายด้าน เช่น GPA จำนวนการขาดเรียน และระยะเวลาในการเรียน

จากการดำเนินงานตั้งแต่การสำรวจข้อมูล การเตรียมข้อมูล ไปจนถึงการสร้างและประเมินผลโมเดล ทำให้เห็นภาพรวมของกระบวนการทำงานด้าน Machine Learning อย่างเป็นระบบ และเข้าใจถึงความสำคัญของแต่ละขั้นตอนในการพัฒนาโมเดลให้มีประสิทธิภาพ

นอกจากนี้ยังทำให้เห็นว่าลักษณะของข้อมูลมีผลต่อการทำงานของโมเดลอย่างชัดเจน โดยเฉพาะในกรณีที่ข้อมูลมีความไม่สมดุล (class imbalance) หรือมีความสัมพันธ์ที่ซับซ้อน ซึ่งส่งผลกระทบต่อความสามารถในการเรียนรู้และการทำนายของโมเดล

โครงการนี้ยังช่วยให้เข้าใจถึงข้อจำกัดของโมเดลแต่ละประเภท รวมถึงความแตกต่างในการเรียนรู้รูปแบบของข้อมูล ทำให้สามารถเลือกแนวทางที่เหมาะสมกับลักษณะของปัญหาได้ดียิ่งขึ้น

โดยสรุป โครงการนี้ไม่เพียงแต่แสดงให้เห็นถึงการนำ Machine Learning มาใช้ในการทำนายผลการเรียนเท่านั้น แต่ยังช่วยเสริมสร้างความเข้าใจในกระบวนการวิเคราะห์ข้อมูล การเลือกใช้โมเดล และการตีความผลลัพธ์ ซึ่งเป็นพื้นฐานสำคัญสำหรับการพัฒนาโครงการด้าน Data Science และ Machine Learning ในอนาคต

8. แหล่งอ้างอิง

[1] Rabie El Kharoua. (2024). *Students Performance Dataset*. [ออนไลน์]. เข้าถึงได้จาก : <https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset> (วันที่ค้นข้อมูล : 2569, 27 เมษายน).

[2] Scikit-learn. (ม.ป.ป.). *Logistic Regression*. [ออนไลน์]. เข้าถึงได้จาก : https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (วันที่ค้นข้อมูล : 2569, 27 เมษายน).

[3] Scikit-learn. (ม.ป.ป.). *Decision Tree Classifier*. [ออนไลน์]. เข้าถึงได้จาก : <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html> (วันที่ค้นข้อมูล : 2569, 27 เมษายน).

[4] Scikit-learn. (ม.ป.ป.). *Random Forest Classifier*. [ออนไลน์]. เข้าถึงได้จาก : <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html> (วันที่ค้นข้อมูล : 2569, 27 เมษายน).

[5] Scikit-learn. (ม.ป.ป.). *Support Vector Machine (SVM)*. [ออนไลน์]. เข้าถึงได้จาก : <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html> (วันที่ค้นข้อมูล : 2569, 27 เมษายน).

[6] อธิพิพงษ์ เขมะเพชร. (2568). เอกสารประกอบการสอนรายวิชา SP452 การเรียนรู้ของเครื่อง. มหาวิทยาลัยหอการค้าไทย.